

Relatorio Tecnico Final

Sistema IoT de Monitoramento de Temperatura e Umidade

Rodrigo Ferreira da Nobrega e Flawbert Lorrان da Silva Costa

Junho de 2026

Informacoes do Projeto

Disciplina: IMD0907 — Tecnologias de Comunicacao para IoT
Instituicao: UFRN - Instituto Metropole Digital
Professora: Fernanda Tasso Salmoria
Alunos: Rodrigo Ferreira da Nobrega e Flawbert Lorrان da Silva Costa
Projeto: <https://github.com/rodrigofnobrege/projeto-iot>
Slides: Diretório docs no github

Conteúdo

1	Introducao	3
2	Objetivos	3
3	Arquitetura	3
3.1	Camada Fisica (firmware)	4
3.2	Camada de Aplicacao (broker)	4
3.3	Camada de Visualizacao (dashboard)	4
4	Protocolo MQTT	4
4.1	O que e MQTT	4
4.2	Topicos	5
4.3	Payload JSON	5
4.4	Qualidade de Servico (QoS)	5
4.5	Reconexao	5
4.6	Comandos uteis	6
5	Validacao de dados	6
6	Deteccao de anomalias	6
7	Controle remoto do dispositivo	7
8	Testes	7
8.1	Descricao dos testes	7
8.2	Resultados obtidos	7
9	Como executar	8
9.1	Opcao 1: Teste rapido sem instalacao local	8
9.2	Opcao 2: Execucao completa em ambiente local	8
9.3	Arquivos principais	9
10	Conclusao	9

1 Introducao

Este projeto implementa um sistema IoT completo de **monitoramento de temperatura e umidade**, da coleta fisica do dado ate a visualizacao em tempo real, usando o protocolo **MQTT** em ambiente simulado pela plataforma Wokwi. O sistema e dividido em tres camadas independentes — sensor, broker e dashboard — que se comunicam exclusivamente por mensagens MQTT.

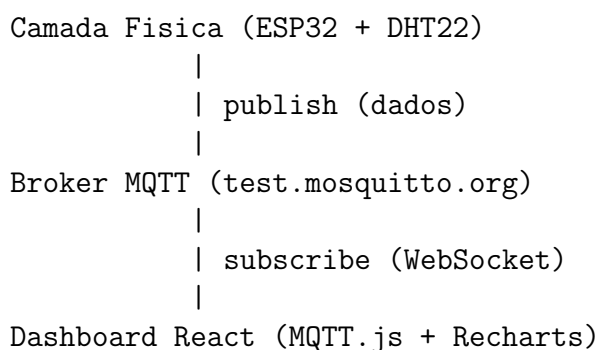
Alem do fluxo de telemetria sensor → dashboard, o sistema oferece **controle remoto**: o dashboard publica comandos em um topico proprio para solicitar uma leitura imediata do microcontrolador.

2 Objetivos

- Coletar temperatura e umidade com um sensor DHT22 acoplado a um ESP32.
- Transmitir as leituras via MQTT, com validacao rigorosa do payload.
- Intermediar a comunicacao por um broker MQTT publico (`test.mosquitto.org`).
- Visualizar os dados em tempo real em um dashboard web, com deteccao de anomalias.
- Permitir o **controle remoto** do dispositivo a partir do dashboard.
- Validar o sistema com uma suite de testes automatizados.

3 Arquitetura

O sistema e organizado em tres camadas que se comunicam exclusivamente por MQTT:



- **Publisher / Subscriber (ESP32)**: le o sensor, valida e publica o payload JSON no topico de dados; e assina o topico de comando para receber solicitacoes do dashboard.
- **Broker**: intermedia todas as mensagens.
- **Subscriber / Publisher (dashboard)**: assina o topico de dados via WebSocket, exhibe os dados em grafico, valida cada mensagem, detecta anomalias e publica comandos.

3.1 Camada Fisica (firmware)

- **Hardware:** ESP32 (DevKit) + sensor DHT22 no **pino GPIO 19**.
- **Linguagem:** **MicroPython** (arquivo `firmware/main.py`).
- **Simulacao:** Wokwi (`firmware/diagram.json`), com WiFi Wokwi-GUEST.
- **Bibliotecas:** `dht`, `network`, `ntptime`, `umqtt.simple`, `json`.

Fluxo principal do `firmware/main.py`:

1. **Conectividade** — conecta ao WiFi e ao broker MQTT; sincroniza o relógio via NTP.
2. **Assinatura de comando** — assina `imd/tec/com/iot/rodflaw/comando`.
3. **Leitura** — le temperatura e umidade do DHT22.
4. **Timestamp** — gera `dataHora` em ISO 8601.
5. **Publicacao** — publica no topico de dados, **a cada 2 segundos**.
6. **Reconexao** — em falha de rede, reconecta ao broker.
7. **Comando** — ao receber comando, dispara leitura imediata.

3.2 Camada de Aplicacao (broker)

- **Broker:** `test.mosquitto.org`
- **Porta 1883:** MQTT sobre TCP (firmware ESP32)
- **Porta 8081:** MQTT sobre WebSocket seguro wss (dashboard navegador)
- **Sem autenticacao:** o broker de testes e aberto

3.3 Camada de Visualizacao (dashboard)

- **Stack:** React + Vite + MQTT.js + Recharts
- **Conexao:** WebSocket `wss://test.mosquitto.org:8081`
- **Componentes:** `StatusBar`, `CurrentValues`, `SensorChart`, `AnomalyAlert`, `DeviceControl`
- **Validacao:** `payload.js`, `anomaly.js`, `format.js`, `useMQTT.js`

4 Protocolo MQTT

4.1 O que é MQTT

MQTT (Message Queuing Telemetry Transport) é um protocolo de mensageria leve no padrão **publish/subscribe**, projetado para dispositivos com recursos limitados. O sensor publica em um **topico**, o broker distribui, e os subscribers que assinam aquele topico recebem as mensagens.

4.2 Topicos

Topico	Direcao	Uso
imd/tec/com/iot/rodflaw	ESP32 → dashboard	leituras do sensor
imd/tec/com/iot/rodflaw/comando	dashboard → ESP32	controle remoto

Tabela 1: Topicos MQTT utilizados

4.3 Payload JSON

```
1 {  
2   "id_dispositivo": "esp32-01",  
3   "temperatura": 25.4,  
4   "umidade": 68.2,  
5   "dataHora": "2026-06-14T10:30:00"  
6 }
```

Regras de validacao:

Campo	Tipo	Regra
id_dispositivo	string	nao vazia
temperatura	float	−40.0 a 85.0 graus C
umidade	float	0.0 a 100.0 %
dataHora	string	ISO 8601

Tabela 2: Regras de validacao do payload

4.4 Qualidade de Servico (QoS)

- **QoS 0:** At most once (sem confirmacao)
- **QoS 1:** At least once (com confirmacao)
- **QoS 2:** Exactly once (handshake completo)

Neste projeto: assinaturas usam QoS 1, publicacoes usam QoS 0.

4.5 Reconexao

Tanto o firmware quanto o dashboard tratam quedas de conexao:

- **Firmware:** reconecta ao broker e reassina topicos
- **Dashboard:** MQTT.js reconecta automaticamente

4.6 Comandos uteis

```
1 # escutar o topico de dados
2 mosquitto_sub -h test.mosquitto.org \
3   -t "imd/tec/com/iot/rodflaw" -v
4
5 # publicar uma leitura de teste
6 mosquitto_pub -h test.mosquitto.org \
7   -t "imd/tec/com/iot/rodflaw" \
8   -m '{"id_dispositivo":"esp32-01",\
9       "temperatura":25.4,"umidade":68.2,\
10      "dataHora":"2026-06-14T10:30:00"}'
11
12 # enviar comando de leitura imediata
13 mosquitto_pub -h test.mosquitto.org \
14   -t "imd/tec/com/iot/rodflaw/comando" -m "1"
```

5 Validacao de dados

A validacao acontece em **dois pontos**:

1. **No firmware**: ao construir o payload, valores sao formatados no padrao JSON obrigatorio antes de publicar.
2. **No dashboard**: ao receber via `validatePayload`, protege a visualizacao de mensagens invalidas.

Regras: `id_dispositivo` nao vazio; `temperatura` em $[-40, 85]$; `umidade` em $[0, 100]$; `dataHora` em ISO 8601. Mensagens invalidas sao **logadas e descartadas**.

6 Deteccao de anomalias

O dashboard mantem uma janela das **ultimas leituras** (`ANOMALY_WINDOW = 20`) e, para cada campo, calcula media e desvio-padrao. Uma nova leitura e marcada como anomalia somente quando satisfaz **dois criterios**:

1. **Criterio estatistico (z-score)**: desviar **mais de 2 desvios-padrao** da media.
2. **Zona morta**: afastar-se da media pelo menos `ANOMALY_MIN_DEVIATION` (3.0 graus C para temperatura, 3.0% para umidade).

O **baseline** exclui leituras ja marcadas como anomalia, permitindo que anomalias persistentes continuem sendo detectadas.

7 Controle remoto do dispositivo

O dashboard permite acionar o ESP32 remotamente via MQTT:

- O componente `DeviceControl` exibe um botao que chama `publishCommand('1')`
- Publica "1" em `imd/tec/com/iot/rodflaw/comando`
- O firmware, inscrito no topico, reconhece o valor "1" e dispara leitura/publicacao imediata
- O botao fica desabilitado sem conexao e exibe feedback de sucesso/falha

8 Testes

A suite em `tests/` (Python + `paho-mqtt`) valida o comportamento do sistema. Cada execucao grava um relatorio em `docs/resultados/*.md`.

Teste	O que valida
<code>test_latency.py</code>	latencia de entrega com 500 mensagens
<code>test_multi_client.py</code>	multiplos subscribers recebendo a mesma msg
<code>test_reconnect.py</code>	recuperacao apos queda e reconexao

Tabela 3: Suite de testes automatizados

8.1 Descricao dos testes

- **Latencia:** publica 500 mensagens sequenciais e mede tempo de ida e volta, reportando media, minima, maxima e P95. Passa se media < 150 ms.
- **Multiplos clientes:** conecta 3 subscribers ao mesmo topico, publica uma mensagem e verifica que **todos** a recebem.
- **Reconexao:** confirma que subscriber recebe antes da queda, derruba conexao de proposito, reconecta e confirma que volta a receber.

8.2 Resultados obtidos

Execucao em 2026-06-21, contra Mosquitto local (`localhost:1883`), QoS 1 — **3/3 testes aprovados:**

Teste	Status	Resumo
<code>test_latency</code>	OK	media = 0.71 ms, P95 = 1.02 ms
<code>test_multi_client</code>	OK	3/3 subscribers receberam
<code>test_reconnect</code>	OK	recebeu antes e depois da queda

Tabela 4: Resultados dos testes

9 Como executar

9.1 Opcao 1: Teste rapido sem instalacao local

A forma mais rapida e pratica de testar a aplicacao e sem necessidade de instalar nada localmente:

1. Acesse <https://wokwi.com>
2. Copie os arquivos `main.py` e `diagram.json` do repositorio para o simulador do Wokwi
3. Inicie a simulacao no Wokwi
4. Enquanto a simulacao roda, acesse o dashboard em: <https://projeto-iot-inky.vercel.app/>
5. O dashboard estara conectado ao broker publico e receberao os dados da simulacao em tempo real

Esta e a opcao ideal para avaliacao rapida do sistema sem configuracoes adicionais.

9.2 Opcao 2: Execucao completa em ambiente local

Para uma execucao completa com controle total do sistema:

```
1 # 1. Firmware - simular no Wokwi ou programar em ESP32 real
2 #   Arquivo: firmware/main.py
3 #   Arquivo de simulacao: firmware/diagram.json
4 #   Aponta automaticamente para test.mosquitto.org
5
6 # 2. Dashboard (em desenvolvimento local)
7 cd dashboard
8 npm install
9 npm run dev
10
11 #   Estara disponivel em http://localhost:5173
12 #   Ja configurado para wss://test.mosquitto.org:8081
13
14 # 3. Testes automatizados
15 #   Suba o broker MQTT local para testes
16 cd mosquitto
17 docker compose up -d
18
19 #   Execute a suite de testes
20 cd ../tests
21 python3 run_tests.py
22
23 #   Relatorio consolidado em: docs/resultados/_resumo.md
```

9.3 Arquivos principais

Arquivo	Descricao
<code>firmware/main.py</code>	Codigo do microcontrolador (MicroPython)
<code>firmware/diagram.json</code>	Configuracao de simulacao do Wokwi
<code>dashboard/src/config.js</code>	Configuracao centralizada do dashboard
<code>tests/run_tests.py</code>	Executavel principal da suite de testes
<code>mosquitto/docker-compose.yml</code>	Configuracao do broker local

Tabela 5: Principais arquivos do projeto

10 Conclusao

O projeto entrega um pipeline IoT funcional e desacoplado: coleta fisica em **MicroPython** no ESP32, transmissao via **MQTT** por um broker publico (`test.mosquitto.org`), visualizacao em tempo real no dashboard React, deteccao de anomalias por media movel e **controle remoto** do dispositivo via topico de comando.

A validacao de dados em ambas as pontas e a suite de testes automatizados demonstram, na pratica, os conceitos de mensageria publish/subscribe, qualidade de servico, comunicacao bidirecional e resiliencia de conexao estudados na disciplina.